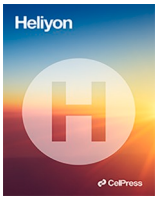
Heliyon 10 (2024) e35959



Contents lists available at ScienceDirect

Heliyon

journal homepage: www.cell.com/heliyon



Research article



Character recognition system for pegon typed manuscript

Yova Ruldeviyani *, Heru Suhartanto, Beltsazar Anugrah Sotardodo, Muhammad Hanif Fahreza, Andre Septiano, Muhammad Febrian Rachmadi

Faculty of Computer Science, Universitas Indonesia, Depok, Jawa Barat, 16424, Indonesia

ARTICLE INFO

Keywords:
Arabic
Deep learning
Character recognition
Pegon
Segmentation

ABSTRACT

The Pegon script is an Arabic-based writing system used for Javanese, Sundanese, Madurese, and Indonesian languages. Due to various reasons, this script is now mainly found among collectors and private Islamic boarding schools (pesantren), creating a need for its preservation. One preservation method is digitization through transcription into machine-encoded text, known as OCR (Optical Character Recognition). No published literature exists on OCR systems for this specific script. This research explores the OCR of Pegon typed manuscripts, introducing novel synthesized and real annotated datasets for this task. These datasets evaluate proposed OCR methods, especially those adapted from existing Arabic OCR systems. Results show that deep learning techniques outperform conventional ones, which fail to detect Pegon text. The proposed system uses YOLOv5 for line segmentation and a CTC-CRNN architecture for line text recognition, achieving an F1-score of 0.94 for segmentation and a CER of 0.03 for recognition.

1. Introduction

Islam in Indonesia spread through integration with local cultures and traditions, utilizing various methods to ensure a lasting impact. This approach involved engaging with local languages, translating religious texts, and establishing a common means of communication through scripts. The adaptation required developing writing scripts compatible with both vernacular and religious languages, such as Arabic. One native script to Indonesia is the Pegon Abjad, historically used for Javanese, Maduranese, and Sundanese languages.

The process of automatic digitization is known as Optical Character Recognition (OCR). It has evolved from recognizing individual characters to translating image sequences into text sequences. An OCR system for Arabic-based script like Pegon faces challenges due to its complex script, diacritics, and ligatures. It must accurately identify 70–80 symbols, including basic letters, diacritic marks, and other symbols, many of which are unavailable in existing Arabic datasets. Compared to similar scripts, Pegon is exceptionally low-resource for digitization.

This research focuses on offline recognition of typed Pegon text, excluding variable texts like medieval manuscripts and modern handwritten Pegon text. It excludes calligraphic and stylized texts and heavily rotated or distorted Pegon texts. The objective is to design and evaluate an OCR system that accurately recognizes and interprets text within line-level boundaries. The research also excludes preprocessing techniques like deskewing or denoising. For this purpose, it introduces datasets to evaluate these systems.

For the text detection module, this research compared three methods for detecting Pegon texts from images: A* algorithm, Disentangled Contour Learning Network (DCLNet), and You Only Look Once ver 5 (YOLOv5) [1–3]. The A* algorithm was selected as a

* Corresponding author.
E-mail address: yova@cs.ui.ac.id (Y. Ruldeviyani).

https://doi.org/10.1016/j.heliyon.2024.e35959
Received 13 March 2024; Received in revised form 5 August 2024; Accepted 6 August 2024
Available online 10 August 2024
2405-8440/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

baseline due to its status as a state-of-the-art method for text segmentation, excluding machine learning and deep learning approaches. DCLNet was chosen based on its superior performance over other deep learning text segmentation methods for documents [4]. YOLOv5 was selected because it represented the state of the art in general object detection at the time of our research and its fast inference time on COCO dataset of around 2.4 ms [5,6].

For the text recognition method, we compared two methods based on Connectionist Temporal Classification - Convolutional Recurrent Neural Network (CTC-CRNN) architecture, a modified version based on our use case of the previously mentioned two methods, and TrOCR, an OCR model based on the Transformers architecture [7–9]. Methods based on CTC-CRNN were chosen because they represented the state-of-the-art OCR models for Arabic text with diacritics, similar to Pegon, at the time of this research. TrOCR was included because it showed better results for English text OCR compared to CTC-CRNN based models, although its performance for Arabic text with diacritics had not yet been established. Therefore, this model was included to explore its potential for this use case.

The paper's structure is outlined as follows: Section 2 presents the literature review; Section 3 elaborates on the methodology; Section 4 covers the result of research and analysis; Section 5 offers conclusions and ends with future research in Section 6.

2. Literature review

2.1. Pegon orthography

The Pegon script is unique among Arabic-derived scripts as it requires the use of harakat to indicate vowels. The exception is pegon gundhul, a more contemporary variant that does not use harakat. The inclusion of harakat is necessary because Javanese, Sundanese, Maduranese, and Indonesian languages use the sounds [ɛ], [ə], and [ɔ] in addition to [a], [i], and [u] found in Arabic. These vowel representations in Pegon are listed in Table 1. For consonants, Pegon introduces several new characters, which, along with their corresponding sounds and contextual forms, are listed in Table 2.

2.2. Typed manuscripts and digital encoding

This research focuses on typed texts for their readability and consistency compared to handwritten ones, as typed texts are produced using standardized type. Another important concept is machine-encoded orthography, specifically the Unicode standard. This encoding is used as the sole reference for both data annotation and model output encoding.

2.3. Data synthesis

Data synthesis techniques, such as the Synthetic Minority Over-sampling Technique (SMOTE), offer a solution for low-resource tasks by generating new data points based on established assumptions [10]. In machine learning pretraining, data synthesis enables training on a more extensive and generalized dataset before fine-tuning on a smaller, more specific one [11]. The applications for OCR on the Pegon abjad should be evident. This research synthesizes two specific forms of data: text images and textures.

2.3.1. Text image synthesis

Text image synthesis involves creating data containing text images, using machine-encoded text as the basic form. Techniques include writing text on images, generating backgrounds, fonts, font settings, randomizing, and skewing. Deep learning approaches often use end-to-end generative methods [12,13]. However, most methods focus on in-the-wild text images, such as signs and streets, rather than historical manuscripts.

2.3.2. Texture synthesis

Texture synthesis is the process of creating new textures that resemble natural or artificial patterns in materials or surfaces. It uses gradient descent or backpropagation on an input sample or seed image [14]. The input image is augmented, filtered, and a loss function is calculated. Optimization algorithms like Limited memory – Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) and Stochastic Gradient Descent (SGD) are used to optimize the image, generating desired output images over several iterations [15].

2.4. Conventional OCR design

Conventional OCR involves sequential stages of processing, including image pre-processing, segmentation, and feature extraction. These stages produce one-dimensional vectors for character classification, which can then be classified into specific classes of

Table 1
Pegon vowels using the letter ب (ba).

بَ / با / بَا	[ba]	بَِ	[bɛ]
بِ / بي / بِي	[bi]	بَو	[bɔ]
بُ / بو / بُو	[bu]	بَ	[bə]

Table 2
Pegon consonants with their corresponding sounds and contextual forms.

Sound	Isolated	Initial	Medial	Final	Variant
[ʧ]	ﭽ	ﭼ	ﭽ	ﭻ	
[d]	ﺪ	N/A	N/A	ﺪ	ﮈ
[g]	ﮓ	ﮕ	ﮕ	ﮓ	ﮒ، ﮔ، ﮓ، ﮕ، ﮒ، ﮓ
[p]	ﭫ	ﭬ	ﭭ	ﭫ	
[ɲ]	ﯼ	ﯾ	ﯿ	ﯽ	ﮋ
[t]	ﻂ	ﻃ	ﻄ	ﻂ	ﻁ، ﻃ
[ŋ]	ﻎ	ﻏ	ﻐ	ﻎ	
[v]	ﻭ	N/A	N/A	ﻮ	

characters. Details of each step are as follows:

Image Preprocessing Enhancing image quality to improve performance in subsequent stages. Examples: Noise reduction (e.g., binarization, thresholding), alignment techniques (e.g., deskewing, cropping, and scaling), connected component analysis.

Segmentation Also known as text detection. Breaking down images into lines (line segmentation), words, character (character segmentation), or other textual components. Example: A* algorithm.

Feature Extraction Transforming raw input into a one-dimensional feature space. This process statistically encodes features like color and line position as classifiable numbers.

Classification Methods used to classify results from feature extraction. Examples: Support Vector Machines (SVM) and Naive Bayes (NB) classifiers.

2.5. OCR with deep learning

Deep learning, also known as Neural Networks, is a statistical learning technique that optimizes data by breaking down features into deep graphs of statistical dependence. This process involves massive training architectures to achieve generality and produce specific models. Deep learning has applications in OCR, particularly in line segmentation and text recognition [16,17].

2.5.1. Deep text line segmentation

Deep text line segmentation or deep text detection uses deep learning algorithms to segment text lines in documents, rather than heuristics or hand-crafted algorithms. This approach is promising due to its robustness to variations in writing, as shown by recent works [18–20]. Deep object detection frameworks such as DCLNet and YOLOv5 are therefore of interest to this research and will be adopted to be evaluated. However, some adaptations are needed as these models are designed for texts in the wild, with larger variations in angle, size, and shape. Details of adoption are discussed in sections 3.2.3 and 3.2.4.

2.5.2. Deep text recognition

Text recognition with deep learning is usually categorized as sequence recognition, a deep learning paradigm that transforms data into another, such as audio transcription or text translation. Two main approaches of sequence recognition for OCR are Connectionist Temporal Classification (CTC) and Sequence-to-Sequence (Seq2Seq) recognition [21].

CTC CTC is a loss function/training framework designed to handle the ambiguity of step boundaries in an unsegmented sequence. For example, in cursive scripts like Arabic, the boundaries between letters are not always clear. CTC works by discretizing the input into time steps aligned with the expected output and modeling the sequence of possible class predictions as a path. The path with the highest product of each timestep is selected, representing the value of loss/distance to the expected output.

CRNN The Convolutional Recurrent Neural Network (CRNN) is a deep learning architecture that consists of a Convolutional Neural Network (CNN) module and an RNN module. The CNN extracts features from the input image, which are then fed into the RNN and optimized through the CTC loss. CRNNs are widely used in CTC-based OCR systems, including for Arabic text. A possible modification for this architecture is to replace the CNN module with a feature extractor module such as EfficientNet [22].

Seq2Seq Seq2Seq is a sequence-to-sequence learning paradigm that uses attention mechanisms to focus on specific parts of input sequences, improving the model's ability to capture dependencies and contextual information. An example is the transformer architecture, which uses self-attention to capture long-range dependencies in sequential data [23].

TrOCR TrOCR is an end-to-end Transformer-based OCR model for text recognition. It employs the Vision Transformer (ViT) architecture, which adapts the transformer/attention mechanism for images [24]. TrOCR offers three advantages: it utilizes pre-trained image and text Transformer models, allowing for large-scale unlabeled data for image understanding and language modeling, and eliminates the need for convolutional networks in the backbone, simplifying implementation and maintenance processes.

3. Methodology

This section discusses the design of the research. First, the datasets used in this research are explained along with their generation process. This is followed by an explanation of the proposed methods adopted in this research. Finally, the design of the experiments to evaluate these methods is described.

3.1. Datasets

There are three datasets used in this research: the Arabic dataset, the Annotated Pegon dataset, and the Synthesized dataset.

3.1.1. Arabic dataset

Prior to this research, there was no sufficiently large dataset of text images in the Pegon Abjad for character recognition tasks. Therefore, supplementary datasets were used. The datasets of interest are images and their annotations of text written in any Arabic script, ideally with harakat present in both the images and annotations. The dataset chosen for this research comprises text images from the Al-Qur'an—the Islamic holy scripture [25].

3.1.2. Annotated Pegon dataset

An annotated Pegon dataset was produced for this research, consisting of annotations from existing physical Pegon texts printed in book form. The dataset includes three books written in Javanese: *Al-Mujarrabat Al-Kubra*, *Mujarrabah*, and the *Majmu'ah Syariah* [26–28]. These books were scanned using an HP DeskJet 2135, digitized into bitmap format at a resolution of 200 DPI (dots-per-inch).

To annotate the dataset, an annotation tool called Labelme was used [29–31]. The annotation process involves marking and labeling each line text of every page image from the dataset. Annotators marked all the line text parts of an image with rectangular bounding boxes and annotated the text using the tool provided by Labelme.

3.1.3. Synthesized dataset

This research also introduces the PegonSynth dataset, a synthesized Pegon text image dataset containing Pegon or Arabic text and JSON files, with its variant the PegonCharSynth dataset. Two methods were evaluated in the synthesis process: the first uses the text data synthesis method SynthTIGER, while the second was developed using ad-hoc methods. The SynthTIGER engine consists of two components: the text selection module and the text rendering module. The ad-hoc synthesizer pipeline involves taking existing Pegon text data and placing it on generated background images that resemble manuscripts. The text data itself was combined from various internet sources and existing digital Pegon texts.

A method for background generation was adopted for the ad-hoc synthesizer using the software package texturize. The backgrounds were generated from screen captures of blank pages or portions of pages of existing manuscripts as seed images. Each set of images generated from the same seed image can be considered a "set," as if belonging to an empty manuscript.

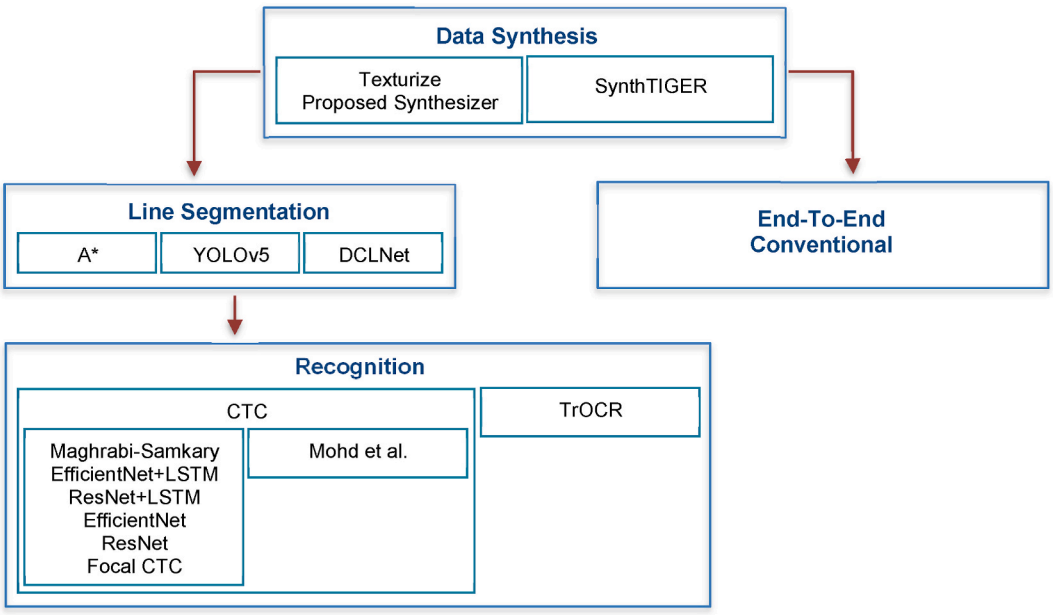


Fig. 1. Overview of the experiments.

3.2. Proposed methods

This section describes the adopted methods and the proposed new methods experimented with during the research. It begins with a discussion on the evaluated segmentation methods and any relevant adaptations, followed by a discussion on the recognition methods and the modifications tested.

In general, this research is divided into two paths: the deep learning research path and the end-to-end conventional path. The deep learning path is divided into two stages: the line segmentation stage and the recognition stage. Various proposed methods are tested and compared in each stage to find the best method. Meanwhile, in the end-to-end conventional path, a non-deep learning method is explored to assess its potential and efficacy against the deep learning methods. An overview of the experiments for the proposed methods can be seen in Fig. 1.

3.2.1. End-to-end conventional method

Of interest for comparison in this research is a conventional end-to-end system that encompasses preprocessing, segmentation into lines, words, and characters, and finally classification [32]. The segmentation process is done in sequential steps, starting with segmenting a page into several lines using a line segmentation algorithm. Each line from the previous step is segmented into possible words using a word segmentation algorithm. Subsequently, each segmented word is segmented into possible characters using character segmentation algorithm.

The classification step is conducted after the possible characters are successfully retrieved from the previous step. The classification step could choose from four different methods: Linear Support Vector Machine, Gaussian Naive Bayes, 1-Layer Neural Network, and 2-Layer Neural Network. Each of these methods were modified by training them using the PegonCharSynth dataset. The pipeline of this method can be seen in Fig. 2.

3.2.2. A* line segmentation method

This method is based on the A* algorithm [1] to detect cropping points in text line extraction. The algorithm uses a priority queue and explores nodes (pixels) with lower total costs. Here, local maxima of cost represent text line regions, as they block the traversable whitespace regions. This implementation is specifically tuned for Arabic line segmentation [33]. Some modifications are made to accommodate the features of the annotated dataset. This method relies on heuristics derived from the analysis of the annotated dataset and might not work as well with other datasets.

The annotated dataset consists of two-page images that need to be split into left and right page images. To achieve this, the image is divided into three equal parts, with the middle part designated as the Region of Interest (RoI) where the page separation line is typically located. The RoI undergoes Otsu binarization to create a binary image. Horizontal projections of this binary image are then analyzed to identify the line with the lowest cumulative value, which is assumed to be the accurate segmentation point between the two pages. This approach leverages the observation that text and its surrounding borders usually have higher values, distinguishing them from the separating region between pages.

Each of the page images needs to be preprocessed before line segmentation. The initial step involves extracting textual content from within and around the text border. This begins with connected component analysis to identify the largest components, which are likely candidates for the text border. Pixels to the left of the image center are then examined to locate the text border component. Once identified, the thickness of the vertical line part of the text border is determined by counting pixels until a background pixel is encountered (Fig. 3).

Next, the image is rotated to correct any skew using a mask that exclusively covers the text border component. The optimal rotation angle is found by maximizing the horizontal projection values of the rotated mask, ensuring the text lines appear straightest. After rotation, the text border component is removed by expanding and updating pixels identified as outliers in horizontal and vertical projections of the mask (Fig. 4).

Accurately segmenting text lines in the annotated dataset, which includes diacritics and dense lines, is achieved through a two-step approach. First, a secondary binarized image is generated to remove diacritics and noise, retaining only prominent ligatures. This enhances image quality for subsequent analysis. Next, horizontal projections of this image are smoothed using a moving average

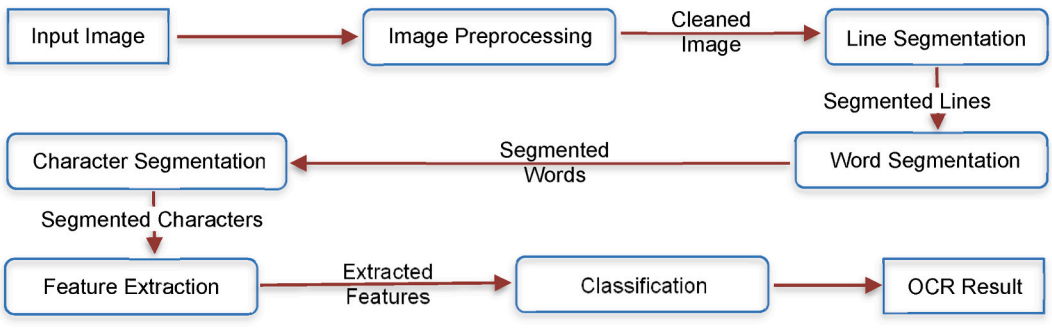


Fig. 2. End-to-end conventional Method pipeline.

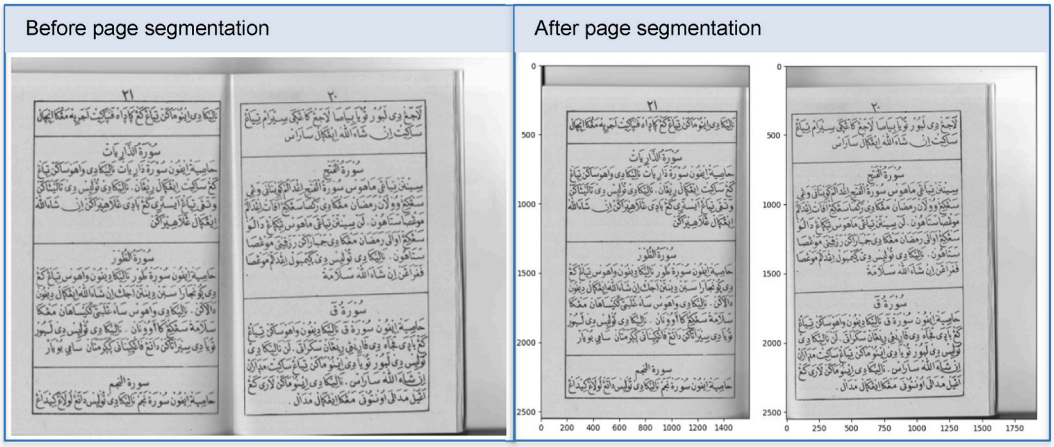


Fig. 3. Image dataset before and after page segmentation.

algorithm to refine accuracy.

To segment text lines, the average height of components in the binarized image approximates line height. Peaks in smoothed projections indicate potential cutting points between lines. Using an A* algorithm, optimal cutting points within identified Regions of Interest (RoIs) are determined. The algorithm starts from edges where horizontal projections are lowest, prioritizing paths with fewer interruptions.

A challenge arises when complete blockages occur within RoIs due to ligatures or diacritics. To overcome this, specific components causing blockages are identified, and modified A* traversal allows navigation around these obstacles, ensuring successful line segmentation. The result of the segmentation can be seen in Fig. 5.

3.2.3. YOLOv5

To adapt YOLOv5 for this specific use case of line text segmentation, the model is fine-tuned on the annotated and synthesized datasets. This ensures that the model learns the specific shapes necessary to be recognized, namely the line segments of the text. The default hyperparameters used in this research can be found in Table 3. The process involves updating the model's weights using the annotated line text images. Only the detected bounding boxes with a confidence ratio greater than 0.5 are chosen as the correct bounding boxes.

3.2.4. DCLNet

An important addition to the DCLNet model pipeline was implemented: a simple detected polygon clustering phase. The DCLNet

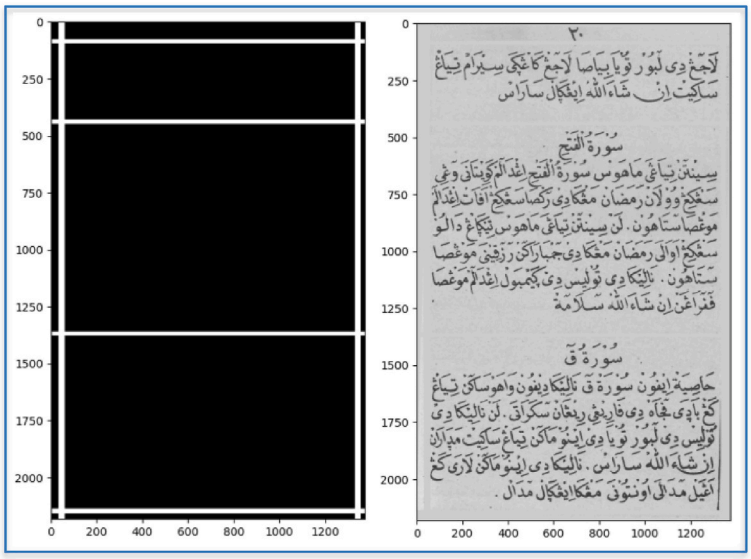


Fig. 4. Modified text border mask and the image with the text border removed.

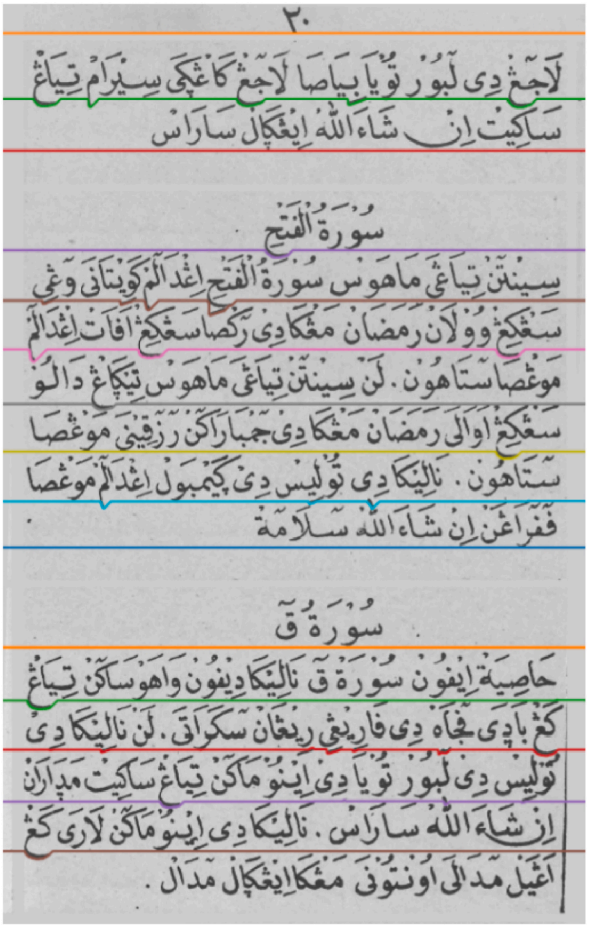


Fig. 5. Result of the line text segmentation using the A* algorithm.

Table 3
YOLOv5 fine-tuning hyperparameters.

Learning Rate	Momentum	Weight Decay	Epoch
0.01	0.937	5E-04	300

model, by itself, segments specific words separated by spaces into polygons instead of whole lines of text. Therefore, the additional phase is intended to group adjacent detected words into one line of text. The method works by measuring the vertical Intersection-over-Union ratio and combining lines with a ratio above a certain threshold. From experimentation, this threshold was chosen to be 0.4.

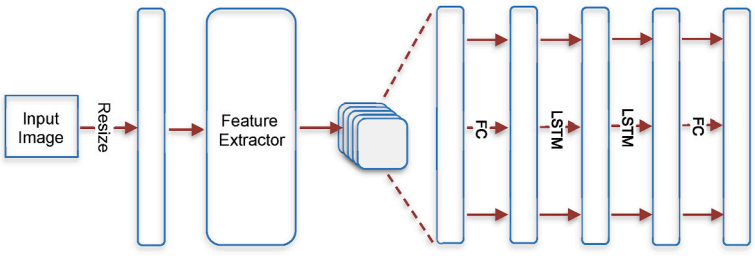


Fig. 6. Diagram of the feature extractor model architecture.

3.2.5. CTC-based recognition methods

CTC loss has been previously proposed for Arabic script recognition. The original method was modified with a feature extractor from an existing pretrained model, such as ResNet or EfficientNet [34,35]. The second modification involved removing the RNN (LSTM) layers, leaving only the feature extractor and classifier layers. This modification was based on the assumption of conditional independence between timesteps of the input sequence, which could lead to better performance. The last modification involves new interpolation operations either on the feature maps from CNN layers to RNN classification or directly on the original image. This adjustment aims to accommodate varying output lengths in models that employ standalone feature extractors, where the output length may not match the intended length. An overview of this architecture can be seen in Fig. 6. Another similar variant used concatenated LSTMs instead of stacked ones to recognize Quranic script.

The hyperparameters of the CTC-based models used in this research can be found in Table 4. Various dropout rates in the range of [0, 0.5] are tested in our experiments. Besides this dropout rate experimentation, the selected hyperparameters are set to the default value of AdamW optimizer.

3.2.6. Transformers-based recognition methods

TrOCR is utilized as the transformer-based model in this research. It is selected because it is specifically designed for OCR tasks, unlike most transformer-based models that are designed for natural language processing tasks or general computer vision tasks. The pre-trained model of the TrOCR is composed of a ViT-based feature extractor and encoder. For encoding and decoding the labels, the model utilizes RobertaTokenize for encoding and AraBERTv2 for decoding [36,37]. This choice is due to the similarities between Arabic and Pegon text.

3.3. Experiment design

This section outlines the methodology employed in assessing the proposed methods discussed in section 3.2. It begins with a discussion on the experiment aimed at evaluating the end-to-end conventional system. Subsequently, it elaborates on the experiments conducted to assess the line segmentation methods. Lastly, it delves into the design of the experiments for text line recognition.

3.3.1. End-to-end conventional system experiment design

The previously mentioned conventional system will be trained on the PegonCharSynth dataset. The dataset is randomly split, with 80 % of the dataset for training and the remaining 20 % for validation. The evaluation is then run on the annotated dataset. The evaluation metric used for the result are the Character Error Rate (Eq. (1)) and the Word Error Rate (Eq. (2)).

$$CER = \frac{n.\ of\ insertions + n.\ of\ deletions + n.\ of\ substitutions}{n.\ of\ characters} \quad \text{Eq. 1}$$

$$WER = \frac{n.\ of\ word\ ins. + n.\ of\ word\ del. + n.\ of\ word\ sub}{n.of\ words} \quad \text{Eq. 2}$$

3.3.2. Segmentation experiment design

The experiment for line text segmentation evaluated both the A* segmentation method and deep learning approaches. The A* segmentation method was evaluated on the entire annotated dataset, while the YOLOv5 method was trained and tested using two datasets: the Pegon annotated dataset and the Pegon synthesized dataset. The YOLOv5 model was assessed in different data scenarios, incorporating both annotated and synthesized datasets. The experimental design was suitable for evaluating the robustness and generalization capabilities of the YOLOv5 model in line text segmentation, while the DCLNet model was evaluated directly due to resource constraints.

Three metrics were used to evaluate the performance of the A* segmentation method and deep learning approaches: precision, recall, and F1 score. These metrics are more suitable for evaluating the performance of models designed to detect a single class, such as text lines, than the more complex mAP metrics. To measure these metrics, the predicted bounding boxes of detected text need to be matched to corresponding ground truth bounding boxes using the Intersection-over-Union (IoU) metric [38].

3.3.3. Recognition experiment design

The experimental design for recognition involves a two-phase train-test split for both CTC-based and TrOCR models: pretraining and finetuning. In the pretraining phase, models are trained on a composite dataset, comprising 70 % PegonSynth dataset and the Quranic Dataset. Evaluation uses two datasets: one with 30 % of each and the other with the entire annotated Pegon dataset. The finetuning phase further trains the models on 50 % of the annotated Pegon dataset and uniformly evaluates the remaining portion at 25 epochs for convergence. Given the small size of the annotated dataset, plain learning methods would be unsuitable, hence the

Table 4
CTC-based models' hyperparameters.

Optimizer	Learning Rate	Dropout Rate	Weight Decay	Epsilon	Epoch
AdamW	0.001	0–0.5	0.01	1E-08	10

introduction of the finetuning phase. This approach allows models to learn effectively after pretraining on larger, mostly synthetic data. The evaluation metrics here are the same as in the end-to-end experiments (Eq. (1) and Eq. (2)). The summarized training setups are presented in Table 5.

4. Results and analysis

This section presents the outcomes of implementing the methodology delineated in section 3. It begins with an investigation into the resulting datasets generated during this research, coupled with an assessment of their similarity and effectiveness, incorporating various combinations utilized for training a baseline model. It then discusses the outcomes of the end-to-end conventional approach. Following this, the section provides an analysis of the results obtained from the line segmentation methods and text recognition methods. Lastly, it conducts a thorough evaluation of the entire pipeline of the proposed system, along with a qualitative analysis of the resulting text aimed at addressing specific system issues.

4.1. Datasets

The datasets previously mentioned comprise three separate datasets. The first is the **annotated dataset**, which was annotated over the course of three months. The dataset comprised three different books totaling 4112 annotated text line images distributed across 112 pages.

The **synthesized dataset** was created from several sources of digital texts. The scraped portion was sourced by querying search engines for websites. This dataset was then cleaned by removing any non-Pegon texts and correcting punctuation, such as trailing and repeated spaces. The general list of sources includes: *Ahadith* from sunnah.com, various blogs such as www.fiqih.co.id, ilmusofa.blogspot.com, mt-syahida.blogspot.com, aksaratube.blogspot.com, and a transcription of *Tarikhul Auliya* by nawaksara.id [39].

Regarding the synthesis results, the first synthesis method failed, particularly in attempting to use various backgrounds for multiline text. On the other hand, the second method, the ad-hoc synthesis, succeeded to varying degrees and was used to generate the final dataset. Two separate datasets were then created: one for single character-diacritic combinations and one for text lines. The languages of the resulting synthesized page images can be seen in Fig. 7. A comparison between the synthesized and annotated dataset can be seen in Fig. 8A and B, respectively.

Furthermore, some statistical analyses can be conducted to see the similarities between the two datasets. As seen in Fig. 9A, the real annotated data is largely grayscale, whereas the synthesized dataset (Fig. 9B) has more color variation. This was done deliberately during synthesis to include a larger superspace of the intended dataset.

Two more statistical measures are formulated to compare the differences between the two datasets. The first is the estimated *non-cursiveness* of the texts in the image, calculated using Eq. (3). The result of applying Eq. (3) can be seen in Fig. 10A for real data and Fig. 10B for synthesized data. The second measure is the mean gradient of the image, which indicates how sharp the image is. These measures are notable because image manipulation and physical scanning produce different results. The gradient formula derives from the Sobel operator, which measures the change in intensity value per grid on either the *x*-axis or *y*-axis, yielding $\frac{\partial I}{\partial x}$ and $\frac{\partial I}{\partial y}$ for an image I, respectively [40]. These two values can be combined into the Laplace operator as in Eq. (4) for a given kernel. The results can be seen in Fig. 11A for the real dataset and Fig. 11B for the synthesized dataset.

$$\textit{Estimated noncursiveness} = \frac{\textit{Number of connected components}}{\textit{Number of characters}} \quad \text{Eq. 3}$$

$$\Delta I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} \quad \text{Eq. 4}$$

From these figures, it can be observed that the synthesized dataset has a wider distribution of non-cursiveness with a smaller mean—i.e., the synthesized images are less sharp. Overall, these figures show that the two datasets have a similar distribution.

4.2. End-to-end conventional results

The results of each model used in the modified conventional method are presented in Table 6. Each model was trained on the PegonSynthChar training set and evaluated on Pegon annotated data. All models achieved a CER of 1 and a WER of 1, indicating that they failed to recognize any characters in the evaluation dataset images. This failure may be due to the synthesized training set being too regular to generalize to the evaluation set, and the models not having sufficient parameters to capture the full data space.

Table 5
Phases of training and datasets used therein. Key: PS = PegonSynth, QD = Quranic Dataset, APD = Annotated Pegon Dataset.

Phase	Training set	Testing set(s)	
Pretrain	70%PS + QDtrain	30%PS + QDtest	100%APD
Finetune	50%APD	50%APD	

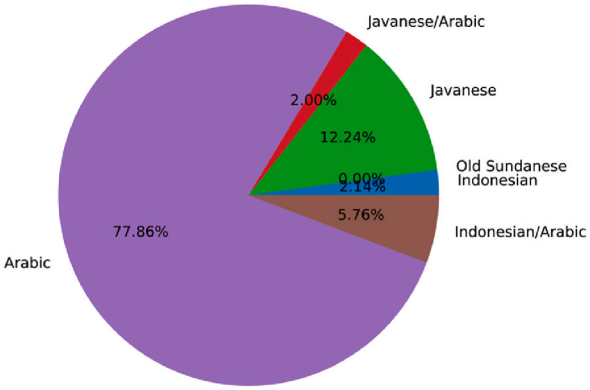


Fig. 7. Percentage of languages of text page image files weighted by the number of lines in a page.

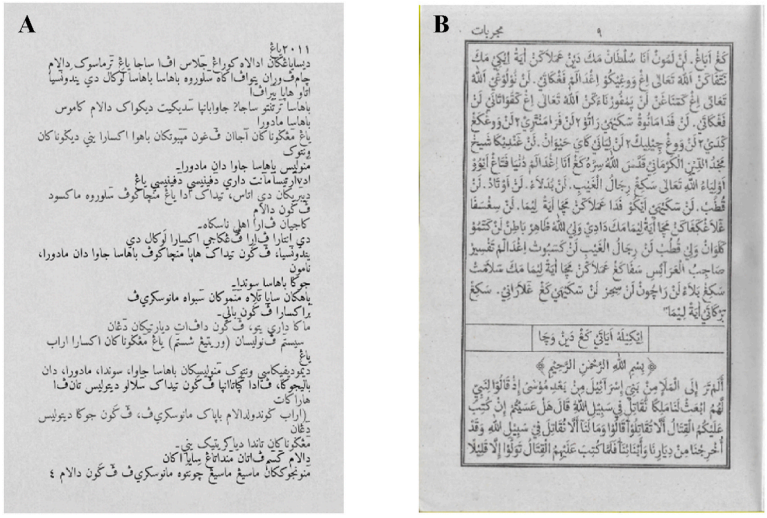


Fig. 8. Comparison of example grayscale images from real data and synthesized data
A Synthesized data with grayscale background
B Real data with grayscale background.

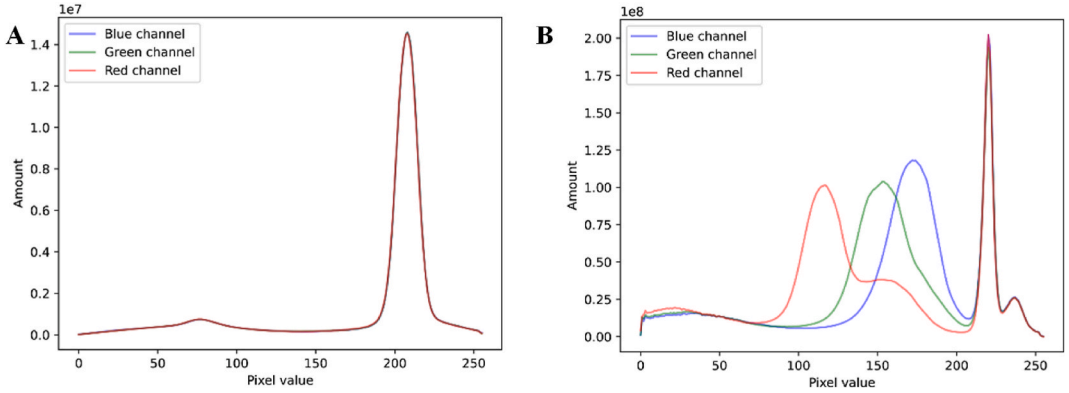


Fig. 9. RGB histograms of the real and synthesized datasets
A RGB histogram of real data
B RGB histogram of synthesized data.

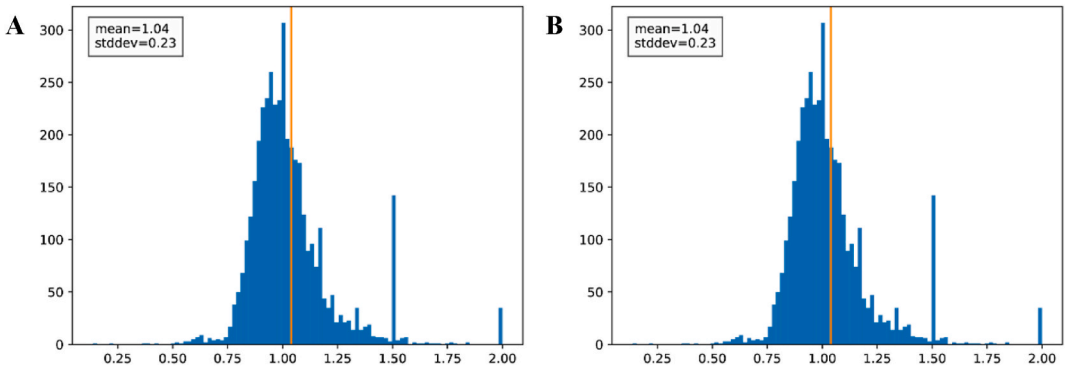


Fig. 10. Estimated non-cursiveness of the synthesized and real dataset
A Estimated non-cursiveness of real data
B Estimated non-cursiveness of synthesized data.

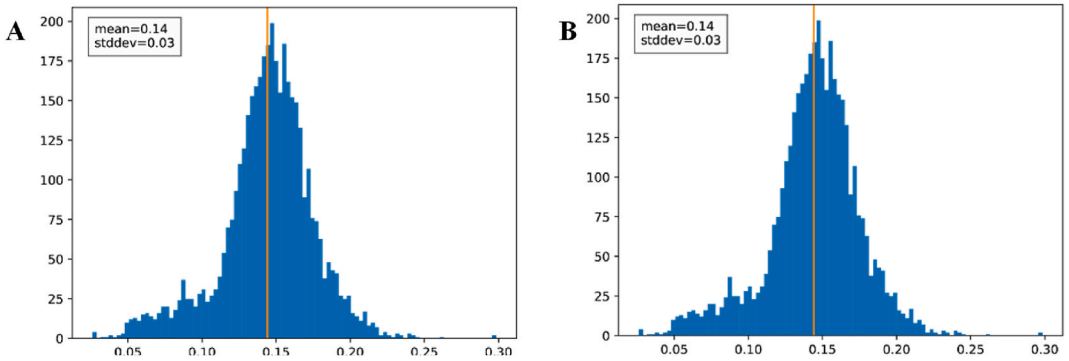


Fig. 11. Mean of nonzero gradients of the real and synthesized dataset
A Mean of nonzero gradients of real data
B Mean of nonzero gradients of synthesized data.

Table 6
Evaluation results of Hussein's conventional method.

Model	CER	WER
Linear SVM	01.00	01.00
Gaussian Naive Bayes	01.00	01.00
One Layer Neural Network	01.00	01.00
Two Layer Neural Network	01.00	01.00

4.3. Line segmentation results

The results of the A* segmentation, DCLNet, and the YOLOv5-based method are shown in Table 7. Each method was evaluated on the respective datasets, with the YOLOv5 method outperforming both the A* segmentation method and DCLNet. A comparison sample image of each method's results with the ground truth label is shown in Fig. 12.

The superior performance of the YOLOv5 method (Fig. 12D), compared to the ground truth segmentation (Fig. 12A), can be attributed to several factors. Beyond the general empirical observation that statistical learning outperforms conventional methods, a specific factor is that the A* segmentation method only generates borders along the y-axis and ignores the x-axis (Fig. 12B). Similarly,

Table 7
Evaluation results of the text line segmentation methods.

Method	Precision	Recall	F1 Score
A* Segmentation	0.73	0.74	0.73
DCLNet	0.90	0.89	0.90
YOLOv5	0.96	0.93	0.94



Fig. 12. Comparison of the tested segmentation methods' results
A Ground truth segmentation
B A* segmentation
C DCLNet detection
D YOLOv5 detection.

DCLNet (Fig. 12C) suffers because its polygon clustering algorithm does not consider the distance between bounding boxes on the x-axis, assuming all texts aligned on the y-axis belong to the same box. This is corroborated by the fact that images where the model exhibited the lowest F1-scores all feature multi-column text.

4.4. CTC methods

4.4.1. Baseline models

The results of the two primary CTC-based methods are summarized in Table 8. Both methods were trained on the PegonSynth and Quranic Dataset training sets and evaluated on the respective testing sets as well as the annotated data. Despite never being trained on the annotated dataset, both models achieved some level of generalization. The results of the fine-tuning phase are noted in the last two columns, showing the same disparity. Note that the WER values remain high due to the lack of word boundary normalization on any of the annotated datasets.

Fig. 13 illustrates the results of using various values for neural dropouts for the M&S model, showing very little effect. This indicates that no regularization is necessary and overfitting has not occurred during model training.

The results of experiments exploring the use of interpolations are noted in Table 9. Using no interpolation seems to yield the best results, although the performance disparity becomes negligible in the final evaluation phase.

4.4.2. Feature extractor models

Fig. 14 A shows the CER and WER rates of the feature extractor models compared to the base model, evaluated on the PegonSynth and Quranic Dataset test sets after training on both respective training sets. In terms of fitting to the training data, the M&S model still outperforms all feature extraction models. However, all models achieve relatively the same level of generalization (Fig. 14B). This

Table 8
Training and evaluation results of the base models.

Model	Test set		Annotated		Finetuned	
	CER	WER	CER	WER	CER	WER
M&S	**0.03**	**0.15**	**0.43**	2.14	**0.15**	**0.62**
Mohd et al.	0.05	0.24	0.6	**1.2**	0.21	0.69

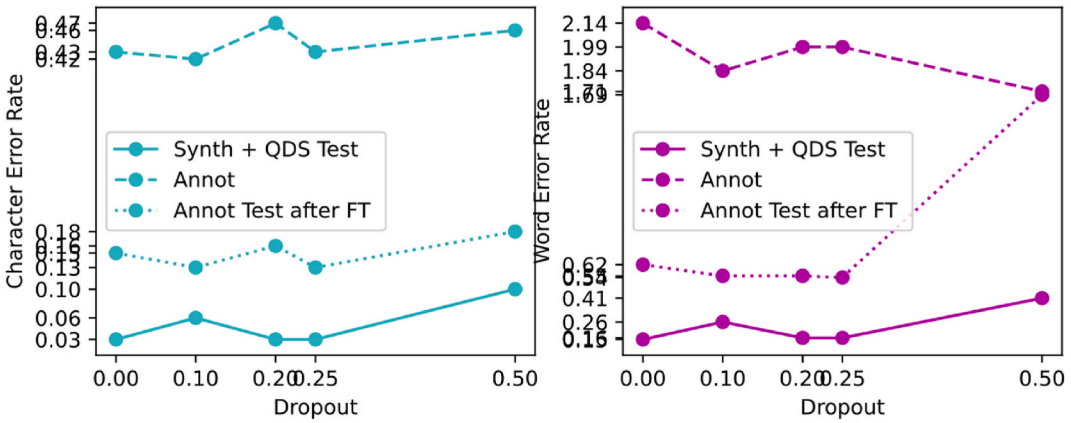


Fig. 13. Results of various values for dropout.

Table 9
Results of different resizing operations.

Input image resizing	Feature map resizing	Test set		Annotated		Finetuned	
		CER	WER	CER	WER	CER	WER
No	No	**0.03**	**0.15**	**0.43**	2.14	**0.15**	**0.62**
No	Yes	0.11	0.45	0.51	1.63	0.19	0.66
Yes	Yes	0.07	0.29	0.47	**1.60**	0.18	0.65

suggests that the synthesized dataset is useful for pretraining these models to similar levels of capability. Notably, larger EfficientNet models perform worse, while larger ResNet models perform better (Fig. 14C).

Although one possible explanation for this is that the latter models are overfitting, this is not the case. Fig. 15 shows that the EfficientNet models (EfficientNetB0, EfficientNetB1, EfficientNetB2) approach convergence towards the same final CER rate with more epochs of training, an effect known as double descent. This occurs as more data or more parameters are introduced [41]. Counting the parameters of the models also supports this explanation (Fig. 16).

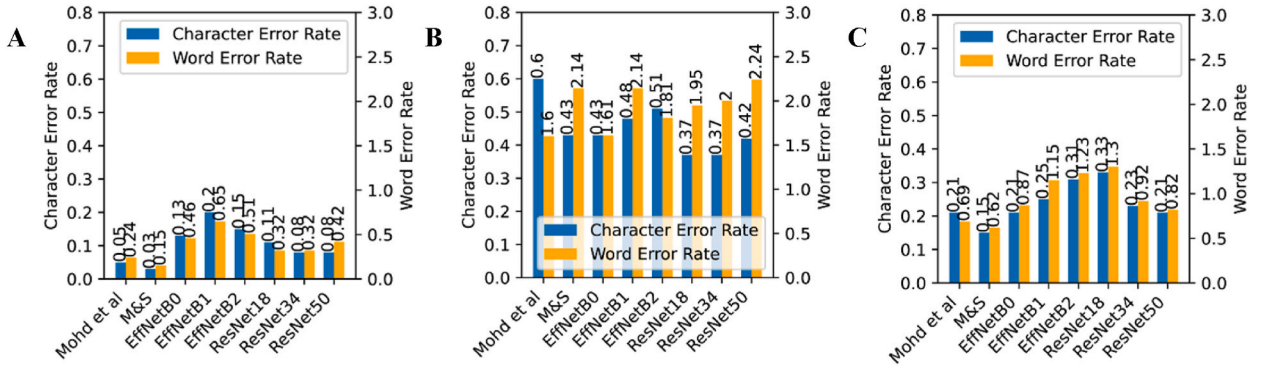


Fig. 14. Results of feature extractor models
A On the corresponding test set
B On unseen annotated data
C After finetuning.

4.4.3. Ablated RNN experiments

Fig. 17A shows the results of removing the RNN layers, which indicate mostly negligible differences. However, generalization suffers as a result, as seen in Fig. 17B. The same curve as in the first figure also appears in Fig. 17C. This indicates that the RNNs provide little utility over the feature maps on seen data but are crucial for generalizability on unseen data.

4.4.4. Ablated dataset experiments

Lastly, Fig. 18 shows the results of training without the Quranic Dataset when evaluated on the annotated dataset, with inconclusively varying levels of improvement or deterioration. Fig. 18A shows the training results on the unseen annotated dataset, and Fig. 18B shows the training results after fine-tuning. The fine-tuning phase results reveal largely negligible effects, likely due to the small size of the dataset compared to the synthesized dataset.

4.5. Transformer method

The TrOCR method was trained for only 3 epochs and yielded a resulting CER of 0.83 on the synthesized dataset. This training process alone took 9 h. The likely cause is the limited data and resources available for training the larger TrOCR model. Given the results up to that point, the experiment was stopped. The TrOCR method did not achieve better results compared to the CTC-based methods.

4.6. Final pipeline and qualitative analysis

This section illustrates an example of the final pipeline from text detection to recognition. The final pipeline is selected from the best-performing results of the line segmentation and recognition sections. Specifically, it utilizes the YOLOv5 method for text detection and the M&S recognition model after fine-tuning. Figs. 19 and 20 display example inputs to the pipeline, with the resulting texts presented alongside them. The bounding boxes of the detected line segmentations were sorted according to the page structure from top to bottom, right to left.

5. Conclusion

This research aimed to provide an initial investigation into the field of Optical Character Recognition (OCR) for Arabic Pegon script. It addressed the unique challenges of recognizing Pegon Arabic script, such as the limited availability of datasets written in the script and its unique diacritical features. Through the analysis of existing literature and experimental results, the following conclusions were reached.

A data synthesis module has been developed for the purposes of synthesizing Pegon text images. An existing text image synthesis tool was evaluated but failed to produce the intended result. It has also been shown that although the synthesized data statistically differs slightly from the real data, it still has utility in training.

Three methods for line text segmentation were tested on the Pegon dataset. First, the A* segmentation method applies heuristics to identify Regions of Interest (RoI) where the correct line for segmenting the texts might be, using the A* algorithm with non-whitespace pixels as obstacles. Second, DCLNet is a deep learning text detection architecture intended for in-the-wild texts at various angles. The specific model used was pretrained on the ICDAR 2017 dataset and the SynthText dataset, with no fine-tuning performed. An ad-hoc algorithm was implemented to adapt its output for documents. Lastly, YOLOv5, the fifth iteration of a deep learning object detection framework, was pre-trained on the COCO dataset. This model was later fine-tuned and tested with the Pegon dataset. The results show that YOLOv5 provides the best result for line-segmenting the Pegon dataset (F1: 0.94), followed by DCLNet (F1: 0.90) and A* (F1: 0.73). Based on these results, YOLOv5 is proposed as the primary method for line segmentation.

The conventional OCR system failed to recognize any characters in the annotated dataset. The probable causes are: the evaluation dataset needed further preprocessing (e.g., border trimming), the training dataset is not compatible with the evaluation dataset, or Pegon script is inherently too complex to recognize. Two CTC-based models with several modifications and one transformer-based model (TrOCR) were tested and compared to determine the most suitable model for the Pegon recognition task. The experiments

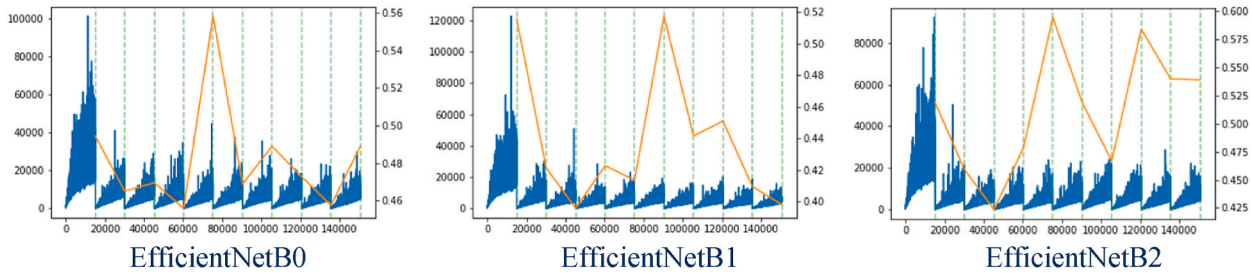


Fig. 15. Graph of training of EfficientNet models, with training loss in blue, epoch markers in green, and CER evaluated on the annotated dataset in orange (EfficientNetB0, EfficientNetB1, EfficientNetB2).

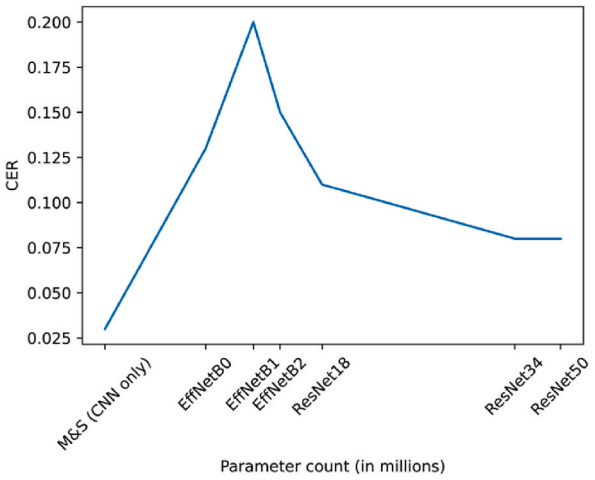


Fig. 16. Parameter count of models plotted against CER.

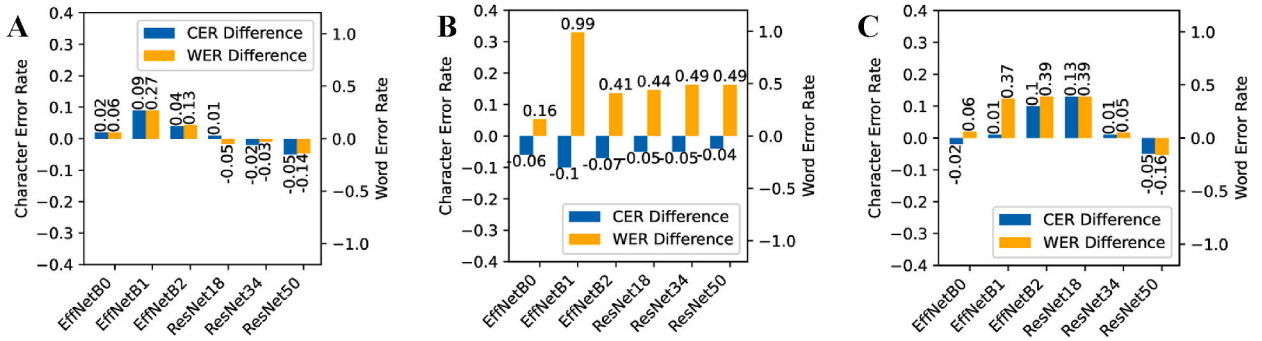


Fig. 17. Change in error rate of feature extraction models with no RNN
A On the corresponding test set
B On unseen annotated data
C After finetuning.

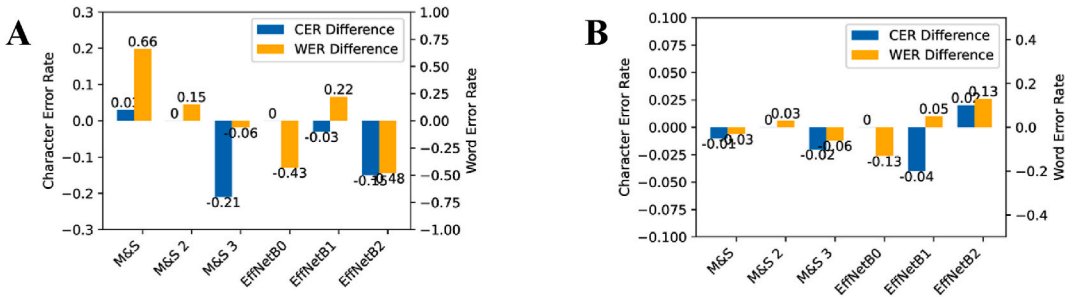


Fig. 18. Change in error rate when the Qur'an Data Set is excluded from training
A On unseen annotated data
B After finetuning.

showed that the CTC model with stacked LSTMs provided the best result in recognizing Pegon line texts, outperforming the model with concatenated LSTMs. The experiments also featured deep double descent and demonstrated that all models showed similar levels of generalization to unseen data. Meanwhile, the TrOCR experiment showed significant underperformance, likely due to a lack of training data, and the experiment was discontinued.

In conclusion, the most suitable components for Pegon OCR, as evaluated in this research, consist of a line segmentation module based on YOLOv5 and a CRNN-CTC text recognition module with stacked LSTMs.

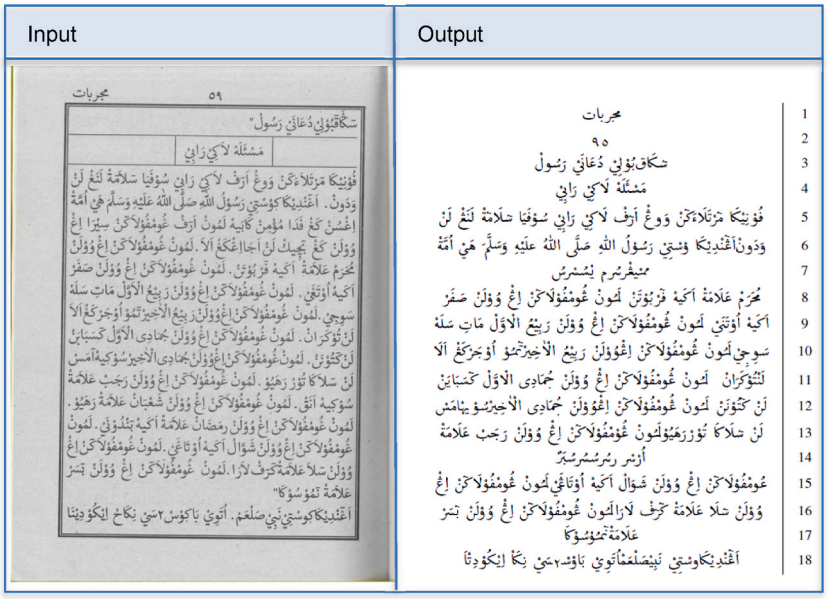


Fig. 19. Input and output of final pipeline (Example 1).

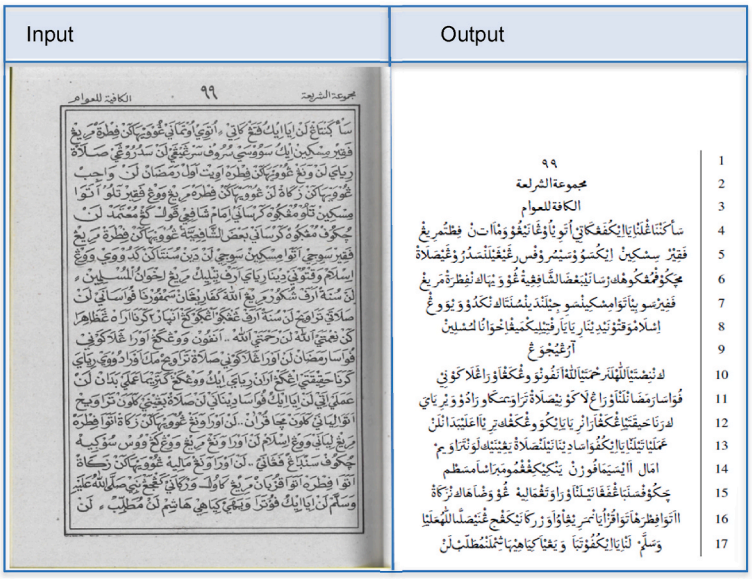


Fig. 20. Input and output of final pipeline (Example 2).

6. Future research

This research focused solely on typed Pegon manuscripts. Future research could explore the following areas: developing an OCR system for handwritten Pegon manuscripts; expanding the annotated Pegon dataset with more diverse sources, expert orthographical evaluation, and annotations that preserve slants and angles; creating a more sophisticated method for synthesizing Pegon text images with skewing, more ligatures, and variable stroke widths; conducting a more comprehensive document layout analysis for historical manuscripts to improve text placement and order accuracy; developing a Pegon OCR system with a transformer-based model trained with sufficient data and resources; and implementing a preprocessing stage capable of handling various levels of deterioration in damaged manuscripts with missing lines.

CRediT authorship contribution statement

Yova Ruldeviyani: Writing – review & editing, Supervision, Project administration. **Heru Suhartanto:** Writing – review & editing, Supervision. **Beltsazar Anugrah Sotardodo:** Writing – original draft. **Muhammad Hanif Fahreza:** Writing – original draft. **Andre Septiano:** Writing – original draft. **Muhammad Febrian Rachmadi:** Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:Yova Ruldeviyani reports financial support was provided by Universitas Indonesia, Faculty of Computer Science. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research did not receive any specific funding.

References

[1] P. Hart, N. Nilsson, B. Raphael, A formal basis for the heuristic determination of minimum cost paths, IEEE Trans. Syst. Sci. Cybern. 4 (2) (1968) 100–107, https://doi.org/10.1109/TSSC.1968.300136.
[2] Y. Bi, Z. Hu, Disentangled Contour learning for quadrilateral text detection, in: 2021 IEEE Winter Conference on Applications of Computer Vision (WACV), IEEE, 2021, pp. 908–917, https://doi.org/10.1109/WACV48630.2021.00095.
[3] Jocher G, Stoken A, Borovec J, et al. ultralytics/yolov5: v3.0. Published online August 13, 2020. Accessed September 26, 2023. https://doi.org/10.5281/zenodo.3983579.
[4] K. Olejniczak, M. Šulc, Text detection forgot about document OCR, in: R. Sablatnig, F. Kleber (Eds.), Proceedings of the 26th Computer Vision Winter Workshop (CVWW 2023), vol. 3349, CEUR Workshop, 2022. https://ceur-ws.org/Vol-3349/paper2.pdf. (Accessed 26 September 2023).
[5] Z. Zou, K. Chen, Z. Shi, Y. Guo, J. Ye, Object detection in 20 Years: a survey, Proc. IEEE 111 (3) (2023) 257–276, https://doi.org/10.1109/JPROC.2023.3238524.
[6] R. Gomez, B. Shi, L. Gomez, et al., ICDAR2017 robust reading challenge on COCO-text, in: 2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR), IEEE, 2017, pp. 1435–1443, https://doi.org/10.1109/ICDAR.2017.234.
[7] A. Maghraby, E. Samkari, Arabic text recognition with harakat using deep learning, IJCSNS International Journal of Computer Science and Network Security 23 (1) (2023) 41–45, https://doi.org/10.22937/IJCSNS.2023.23.1.6. (Accessed 26 September 2023).
[8] M. Mohd, F. Qamar, I. Al-Sheikh, R. Salah, Quranic optical text recognition using deep learning models, IEEE Access 9 (2021) 38318–38330, https://doi.org/10.1109/ACCESS.2021.3064019.
[9] M. Li, T. Lv, J. Chen, et al., TrOCR: transformer-based optical character recognition with pre-trained models, in: The 37th Association for the Advancement of Artificial Intelligence Conference (AAAI 2023), 2021. https://www.microsoft.com/en-us/research/publication/trocr-transformer-based-optical-character-recognition-with-pre-trained-models/. (Accessed 28 September 2023).
[10] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, SMOTE: synthetic minority over-sampling technique, J. Artif. Intell. Res. 16 (1) (2002) 321–357, https://doi.org/10.1613/jair.953.
[11] Y. Wu, F. Li, P. Liang, Insights into pre-training via simpler synthetic tasks, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), *Advances In Neural Information Processing Systems 35*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022, pp. 21844–21857, https://doi.org/10.48550/arXiv.2206.10139.
[12] M. Yim, Y. Kim, H.C. Cho, S. Park, SynthTIGER: synthetic text image GEneratoR towards better text recognition models. 16th International Conference on Document Analysis and Recognition (ICDAR 2021), 2021, pp. 109–124. Published online July 20, http://arxiv.org/abs/2107.09313. (Accessed 26 September 2023).
[13] E. Belval, S. Haozhe, J. Coutault, A synthetic data generator for text recognition, Published online May 10, 2020, https://github.com/Belval/TextRecognitionDataGenerator. (Accessed 23 September 2023).
[14] texture·design. texturize (Version 0.13), GitHub, 2023. Published online April 1, https://github.com/texturedesign/texturize (Accessed 20 September 2023).
[15] D.C. Liu, J. Nocedal, On the limited memory BFGS method for large scale optimization, Math. Program. 45 (1–3) (1989) 503–528, https://doi.org/10.1007/BF01589116.
[16] PaddlePaddle. PaddleOCR (Version 2.6.0), GitHub, 2022. Published online August 24, https://github.com/PaddlePaddle/PaddleOCR (Accessed 18 September 2023).
[17] tesseract-ocr. tesseract (Version 5.3.1), GitHub, 2023. Published online April 2, https://github.com/tesseract-ocr/tesseract (Accessed 26 June 2023).
[18] T. Grüning, G. Leifert, T. Strauß, J. Michael, R. Labahn, A two-stage method for text line detection in historical documents, Int. J. Doc. Anal. Recogn. 22 (3) (2019) 285–302, https://doi.org/10.1007/s10032-019-00332-1.
[19] M. Boillet, C. Kermorvant, T. Paquet, Multiple document datasets pre-training improves text line detection with deep neural networks, in: 2020 25th International Conference on Pattern Recognition (ICPR), IEEE, 2021, pp. 2134–2141, https://doi.org/10.1109/ICPR48806.2021.9412447.
[20] A. Islam, T. Anjum, N. Khan, Line extraction in handwritten documents via instance segmentation, Int. J. Doc. Anal. Recogn. 26 (3) (2023) 335–346, https://doi.org/10.1007/s10032-023-00438-7.
[21] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber, Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks, in: Proceedings of the 23rd International Conference on Machine Learning - ICML '06, ACM Press, 2006, pp. 369–376, https://doi.org/10.1145/1143844.1143891.
[22] D.V. Sang, N.H. Thuan, An EfficientNet-like feature extractor and focal CTC loss for image-base sequence recognition, in: 2020 7th NAFOSTED Conference on Information and Computer Science (NICS), IEEE, 2020, pp. 326–331, https://doi.org/10.1109/NICS51282.2020.9335861.
[23] A. Vaswani, N. Shazeer, N. Parmar, et al., Attention is all You need, in: U. von Luxburg, I. Guyon (Eds.), Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS 2017), Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2017, in: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf. (Accessed 27 September 2023).
[24] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., An image is worth 16x16 words: transformers for image recognition at scale, in: The Ninth International Conference on Learning Representations (ICLR 2021), 2021. https://openreview.net/forum?id=YicbFdNTTy. (Accessed 28 September 2023).
[25] I. Al-Sheikh, M. Mohd, A quranic dataset for text recognition, in: L. Warlina, H. Lida, M. Mohd (Eds.), Proceedings of the Proceedings of the 1st International Conference on Informatics, Engineering, Science and Technology, INCITEST 2019, EAI, Bandung, Indonesia, 2019, https://doi.org/10.4108/eai.18-7-2019.2287842, 2019.
[26] A.A.H.Z. Anwar. Al-Mujarrabat Al-Kubra fii Dzikr Khawas Kalam Rabb Al-WariToha Putra, Semarang, 1974.

[27] Rahman A. (Ed.), Mujarrabah, Toha Putra, Semarang.
[28] S. Darat. Majmu'ah Asy-syari'ah Al-kafiyah Lil Awami, Toha Putra, Semarang, 1929.
[29] K. Wada, Labelme - the flexible image labeling, Published online November 20, https://github.com/wkentaro/labelme, 2022. (Accessed 28 September 2023).
[30] A. Torralba, B.C. Russell, J. Yuen, LabelMe: online image annotation and applications, Proc. IEEE 98 (8) (2010) 1467–1484, https://doi.org/10.1109/JPROC.2010.2050290.
[31] B.C. Russell, A. Torralba, K.P. Murphy, W.T. Freeman, LabelMe: a database and web-based tool for image annotation, Int. J. Comput. Vis. 77 (1–3) (2008) 157–173, https://doi.org/10.1007/s11263-007-0090-8.
[32] H. Youssef, H. Osama, M. Magdy, Arabic-OCR, Published online, https://github.com/HusseinYoussef/Arabic-OCR, 2022. (Accessed 28 September 2023).
[33] M. Ayesh, K. Mohammad, A. Qaroush, S. Agaian, M. Washha, A robust line segmentation algorithm for Arabic printed text with diacritics, Electron. Imag. 29 (13) (2017) 42–47, https://doi.org/10.2352/ISSN.2470-1173.2017.13.IPAS-204.
[34] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, 2016.
[35] M. Tan, Q.V. Le, EfficientNet: rethinking model scaling for convolutional neural networks, in: K. Chaudhuri, R. Salakhutdinov (Eds.), *Proceedings Of the 36 Th International Conference On Machine Learning*. Proceedings of Machine Learning Research, 2019, pp. 6105–6114, in: https://proceedings.mlr.press/v97/tan19a.html. (Accessed 9 March 2024).
[36] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, RoBERTa: A robustly optimized BERT pretraining approach, ArXiv (2019) abs/1907.11692.
[37] W. Antoun, F. Baly, H. Hajj, AraBERT: transformer-based model for Arabic language understanding, in: N. Calzolari, F. Béchet, P. Blache, et al. (Eds.), Proceedings of the Twelfth Language Resources and Evaluation Conference, European Language Resources Association, 2020.
[38] P. Jaccard, Etude de la distribution florale dans une portion des Alpes et du Jura, Bull. Soc. Vaud. Sci. Nat. 37 (142) (1901) 547–579.
[39] B. Musthafa, *Tarikhul Auliya.* Menara Kudus, 1952.
[40] I. Sobel, G. Feldman, in: A.I. Presentation at Stanford, Project (Eds.), An Isotropic 3×3 Image Gradient Operator, Academic Press, 1968, pp. 1–5, 2014.
[41] P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, I. Sutskever, Deep double descent: where bigger models and more data hurt, J. Stat. Mech. Theor. Exp. 2021 (2021), https://doi.org/10.1088/1742-5468/ac3a74.